

SISTEMAS EMBEBIDOS AVANZADOS II · DSI · EIE · FCEIA · UNR

Build systems y FreeRTOS en el ESP32-S3

Del botón Build a un RTOS en dos núcleos

```
$ arm-none-eabi-gcc -mcpu=cortex-m0plus -mthumb -T mkl46z256.ld -Wl,--gc-sections ...
```

Prof. Ing. Daniel M. Márquez

Octubre 2026

Rev 1.0

AGENDA

Del botón Build a FreeRTOS en dos núcleos

Tiempo	Bloque
0–5'	Qué hace el botón Build
5–50'	Build system en la KL46Z: etapas, toolchain, startup, linker script, Make y CMake · demos 1 y 2
50–70'	Caso real: ESP-IDF · CMake, componentes y Kconfig · demo 4: el tick de FreeRTOS
70–115'	FreeRTOS en el ESP32-S3: app_main, dos núcleos, diferencias con la KL46Z · demo 5: ejemplos
115–120'	Cierre y material para trabajar en casa

MOTIVACIÓN

¿Qué pasa cuando apretamos Build?

Cinco programas, un script que describe la memoria del chip y una docena de flags. El IDE lo esconde; hoy lo hacemos a mano y después lo automatizamos.

MOTIVACIÓN

El botón Build que ya usaron deja todo a la vista

```
# un proyecto de FreeRTOS_Workspace, carpeta Debug/
makefile · sources.mk · objects.mk  ← Make
source/subdir.mk ...
<proyecto>_Debug.ld            ← linker script
<proyecto>_Debug_memory.ld
<proyecto>.map                 ← mapa de memoria
<proyecto>.axf                 ← el ELF
```

MCUXpresso **genera** el build: makefiles, linker script y startup.

Hoy lo hacemos **nosotros**, con cinco archivos, para entender cada pieza.

OBJETIVO

¿Para qué entender el build si el IDE lo hace solo?

Diagnosticar

Errores de link, RAM que no alcanza, HardFault por stack, el depurador que salta líneas.

Controlar la memoria

Qué ocupa FLASH y qué RAM. El heap y los stacks de FreeRTOS salen de la RAM.

Ubicar el código

Bootloader y aplicación, actualizaciones OTA, datos en una dirección fija.

Entender la optimización

Debug y Release no son iguales: el optimizador cambia el código. Por eso existe volatile.

No depender del IDE

MCUXpresso, CubeIDE, ESP-IDF y Zephyr usan el mismo gcc, el mismo linker y CMake.

Trabajar en la industria

Builds reproducibles, integración continua y trazabilidad del binario.

CONCEPTOS

El IDE no compila

Pieza	Qué hace	KL46Z	ESP32-S3
IDE	Editor, botones y depuración. Llama a las demás piezas	MCUXpresso	VS Code
Build system	Decide qué compilar, en qué orden y con qué opciones	make	cmake + ninja
Toolchain	Traduce y linkea: es el compilador	arm-none-eabi-gcc	xtensa-esp32s3-elf-gcc
Depurador	Graba el chip, breakpoints, variables	gdb + OpenSDA	gdb + USB-JTAG

botón Build → build system → arm-none-eabi-gcc → blinky.elf → depurador → placa

Build es el proceso; **build system** es la herramienta que lo automatiza. Hoy hacemos el build a mano y después lo automatizamos.

ETAPAS DEL BUILD

Del .c al .bin en cinco pasos



Etapa	Comando	Qué produce
Preprocesar	gcc -E	C sin #include ni #define (unas 100 líneas)
Compilar	gcc -S	Assembler Thumb del Cortex-M0+
Ensamblar	gcc -c	Objeto relocalizable: sin direcciones finales
Linkear	gcc -T script.ld	ELF con direcciones, símbolos y debug
Convertir	objcopy -O binary	Imagen cruda, byte a byte, para la FLASH

TOOLCHAIN CRUZADO

Compilamos en la PC, ejecutamos en el Cortex-M0+

arm-none-eabi-gcc

arm · arquitectura del target

none · sin sistema operativo

eabi · ABI embebida: convención de llamadas, alineación, formato de objetos

Flags del CPU

-mcpu=cortex-m0plus
-mthumb

Tienen que ir en la compilación **y** en el link: el linker elige la versión de libgcc según el CPU.

Host x86_64 ≠ target ARMv6-M. El binario no se puede ejecutar en la PC.

DEMO 1 · A MANO, SIN BUILD SYSTEM

El build completo en siete comandos

```
mkdir tmp
set CPU=-mcpu=cortex-m0plus -mthumb
arm-none-eabi-gcc %CPU% -E src/main.c -o tmp/main.i
arm-none-eabi-gcc %CPU% -Os -S src/main.c -o tmp/main.s
arm-none-eabi-gcc %CPU% -Os -c src/main.c -o tmp/main.o
arm-none-eabi-gcc %CPU% -Os -c startup/startup_mkl46z4.c -o tmp/startup.o
arm-none-eabi-gcc %CPU% -T linker/mkl46z256.ld -nostdlib tmp/main.o tmp/startup.o -o tmp/blinky.elf
arm-none-eabi-objcopy -O binary tmp/blinky.elf tmp/blinky.bin
```

Nosotros hacemos de build system: elegimos el orden, los archivos y las opciones.

Probar: sacar -nostdlib. El link falla con *undefined reference to __bss_start__*: entra el startup de newlib, que espera otros símbolos.

DEMO 1 · LO QUE QUEDÓ EN TMP/

Cada etapa deja un archivo distinto

Archivo	Qué es	Para qué sirve
.c	Código fuente	Lo que escribimos
.i	C preprocesado: sin #include ni #define	Ver qué recibe el compilador
.s	Assembler del Cortex-M0+	Ver qué instrucciones se generaron
.o	Código máquina sin direcciones finales	Pieza intermedia, uno por cada .c
.elf	Programa completo con direcciones y símbolos	Depurar (en MCUXpresso: .axf)
.bin	Imagen cruda de la FLASH	Grabar en el chip
.map	Informe del linker: dónde quedó cada cosa	Analizar el uso de FLASH y RAM

STARTUP

Lo que pasa antes de main()

1

Tabla de vectores

En 0x0: el SP inicial y la dirección de Reset_Handler. El hardware los lee al salir de reset.

2

Flash config

En 0x400, propio de Kinetis. FSEC = 0xFE deja el chip sin proteger. Un valor erróneo lo puede bloquear.

3

Reset_Handler

Apaga el watchdog COP, copia .data de FLASH a RAM y pone .bss en cero.

4

main()

Recién acá empieza nuestro código, con las variables globales ya inicializadas.

LINKER SCRIPT

El linker script describe el chip

```
MEMORY
{
  VECTORES (rx) : ORIGIN = 0x00000000, LENGTH = 0x400
  FLASHCFG (rx) : ORIGIN = 0x00000400, LENGTH = 0x10
  FLASH (rx) : ORIGIN = 0x00000410, LENGTH = 256K - 0x410
  RAM (rwx) : ORIGIN = 0x1FFFE000, LENGTH = 32K
}
/* ... */
_sidata = LOADADDR(.data);
.data :
{
  _sdata = .; *(.data*) _edata = .;
} > RAM AT> FLASH
```

MEMORY

Qué memorias hay, dónde empiezan y cuánto miden.

> RAM AT> FLASH

Vive en RAM, se guarda en FLASH.

_sdata, _edata...

Símbolos que el startup usa para saber qué copiar.

AUTOMATIZAR EL BUILD

Quién reemplaza lo que hicimos a mano

Quién	Lee	Qué hace
Nosotros	—	Escribimos cada comando de gcc y decidimos el orden (demo 1)
make	Makefile	Ejecuta esos mismos comandos, solo los necesarios: compara fechas
cmake	CMakeLists.txt	No compila: elige toolchain y opciones y escribe las instrucciones para otro
ninja	build.ninja	Ejecuta las instrucciones que escribió cmake, como make pero más rápido

make y ninja ejecutan; cmake genera. Y todos terminan llamando a arm-none-eabi-gcc.

MAKE

Make es un grafo de dependencias con fechas

```
objetivo: prerequisitos
<TAB>receta

main.o: main.c
<TAB>gcc -c main.c -o main.o
```

Si algún prerequisito es **más nuevo** que el objetivo, Make ejecuta la receta.

Si no, no hace nada. Eso es el **build incremental**.

La receta va indentada con **TAB**, no con espacios.

DEMO 2 · BUILD INCREMENTAL

Make recompila solo lo que cambió

Comando (cmd)	Qué hace
make	Compila los dos .c, linkea, genera el .bin
make	Nada: todo está al día
copy /b src\main.c +,, make	Actualiza la fecha de main.c: recompila solo main.c y linkea
make DEBUG=1	Nada: es la trampa
make clean make DEBUG=1	Ahora sí: compila en Debug (-O0 -g3)
make size	Uso de FLASH y RAM: text 472 en Debug

DEMO 2 · LA TRAMPA

Debug y Release: el nivel de optimización

Release · make

-Os -g

Optimiza el tamaño: reordena y elimina código. Es lo que se graba en el producto. 380 bytes.

Debug · make DEBUG=1

-O0 -g3

Sin optimizar y con toda la información de depuración: breakpoints y variables tal cual. 472 bytes.

Make compara fechas, no opciones

Los .c no cambiaron, entonces no recompila: el .elf sigue siendo Release. Con Make, al cambiar de modo hay que hacer make clean. CMake lo resuelve con una carpeta por modo.

CMAKE + NINJA

CMake no compila: genera el build



1. Configurar · una vez por directorio de build

```
cmake -B build -G Ninja -DCMAKE_TOOLCHAIN_FILE=cmake/arm-none-eabi.cmake -DCMAKE_BUILD_TYPE=Release
```

2. Compilar · cada vez

```
cmake --build build
```

DEMO 3 · DEBUG Y RELEASE

El CMakeLists del blinky

```
add_executable(blinky.elf src/main.c startup/startup_mkl46z4.c)
target_compile_options(blinky.elf PRIVATE
    -mcpu=cortex-m0plus -mthumb -ffunction-sections -fdata-sections)
target_link_options(blinky.elf PRIVATE
    -mcpu=cortex-m0plus -mthumb -T ${LDSCRIPT} -nostdlib -Wl,--gc-sections)
set(CMAKE_C_FLAGS_DEBUG "-O0 -g3")
set(CMAKE_C_FLAGS_RELEASE "-Os -g")
add_custom_command(TARGET blinky.elf POST_BUILD
    COMMAND ${CMAKE_OBJCOPY} -O binary blinky.elf blinky.bin)
```

Demo: configurar build-release y build-debug en paralelo. Mismo fuente, dos .elf: 380 y 472 bytes de text.

PANORAMA

CMake es hoy el estándar en los SDK de MCU

SDK / framework	Build system
NXP MCUXpresso SDK	CMake para GCC, además de proyectos de IDE
Zephyr RTOS	CMake + Kconfig + devicetree, invocado con west · lo vimos
Raspberry Pi Pico SDK	CMake
Espressif ESP-IDF	CMake + Kconfig, invocado con idf.py
ST STM32CubeMX	Puede generar el proyecto en CMake

Lo que vimos hoy con cinco archivos es lo mismo que hacen estos SDK con miles.

CASO REAL · ESP-IDF EN EL ESP32-S3

idf.py es un envoltorio de CMake + Ninja



Componentes

El proyecto es un componente más: FreeRTOS, drivers y log son componentes del SDK.

Linker script generado

No se escribe a mano: el build lo arma con fragmentos que aporta cada componente.

Kconfig

menuconfig decide qué entra en el firmware y cómo se configura.

CASO REAL · ESP-IDF EN EL ESP32-S3

El mismo blink, ahora como componente

FRDM-KL46Z · bare-metal

```
# CMakeLists.txt
add_executable(blinky.elf
  src/main.c
  startup/startup_mkl46z4.c)
target_link_options(blinky.elf PRIVATE
  -T ${LDSCRIPT} ...)

# startup y linker script:
# los escribimos nosotros
```

ESP32-S3 · ESP-IDF

```
# CMakeLists.txt
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(ESP32_FreeRTOS_blink)
# main/CMakeLists.txt
idf_component_register(
  SRCS "blink_example_main.c")
# main/idf_component.yml
dependencies:
  espressif/led_strip: "^3.0.0"
```

No describimos un ejecutable: registramos un **componente**. El **gestor de componentes** descarga led_strip a managed_components/ en el primer build.

CASO REAL · ESP-IDF EN EL ESP32-S3

Kconfig: del menú al #define

Archivo	Rol	Pin del LED
main/Kconfig.projbuild	Define la opción	config BLINK_GPIO
sdkconfig.defaults.esp32s3	Valores iniciales	CONFIG_BLINK_GPIO=48
sdkconfig	Lo que elige menuconfig	CONFIG_BLINK_GPIO=48
build/config/sdkconfig.h	Macro generada	#define CONFIG_BLINK_GPIO 48
blink_example_main.c	Usa la macro	BLINK_GPIO

Caso real: en mi PC el LED andaba, en el repositorio no

El .gitignore excluye sdkconfig. Al clonar, se regenera desde sdkconfig.defaults, que decía 38. Lo que no está versionado, no existe.

DEMO 4 · ESP32-S3

El build también configura el RTOS

```
# sdkconfig (menuconfig › Component config › FreeRTOS › Kernel)
CONFIG_FREERTOS_HZ=100

# FreeRTOSConfig.h (dentro de ESP-IDF, no se edita)
#define configTICK_RATE_HZ  CONFIG_FREERTOS_HZ
```

CONFIG_FREERTOS_HZ	Ejemplo 2: ticks entre mensajes de 1 s
100	100
1000	1000

El mismo fuente, otro sistema operativo: **el tick lo decide el build**, no el código.

FREERTOS EN EL ESP32-S3

app_main() ya corre dentro de una tarea

FRDM-KL46Z · lo que ya conocen

```
int main(void)
{
   xTaskCreate(tarea_led, "led",
      configMINIMAL_STACK_SIZE,
      NULL, 1, NULL);
   vTaskStartScheduler();
   for (;;);
}
```

ESP32-S3 · ESP-IDF

```
void app_main(void)
{
   xTaskCreate(tarea_led, "led",
      4096,  // en BYTES
      NULL, 2, NULL);
   // sin vTaskStartScheduler()
}  // puede retornar
```

ROM → bootloader → ESP-IDF arranca el planificador en los dos núcleos → crea **main_task** (prioridad 1, núcleo 0) → que llama a **app_main()**.

FREERTOS EN EL ESP32-S3

Dos núcleos, un solo kernel (SMP)

Núcleo 0 · PRO_CPU

main_task, Tmr Svc, esp_timer, IDLE0. Wi-Fi y Bluetooth, si se usan.

Núcleo 1 · APP_CPU

IDLE1 y el resto de la aplicación.

```
xTaskCreate(...)              // cualquier núcleo
xTaskCreatePinnedToCore(..., 1)  // solo el núcleo 1
```

Cada núcleo elige su tarea lista de mayor prioridad: **puede haber dos tareas ejecutándose a la vez**.

Deshabilitar interrupciones no alcanza: **taskENTER_CRITICAL(&spinlock)** excluye al otro núcleo.

FREERTOS EN EL ESP32-S3

Mismo FreeRTOS, seis diferencias

Aspecto	KL46Z · FreeRTOS estándar	ESP32-S3 · IDF FreeRTOS
Núcleos	Uno	Dos (SMP)
Inicio	main() y vTaskStartScheduler()	app_main(), planificador ya iniciado
Stack de xTaskCreate	En palabras	En bytes
Tareas IDLE	Una	Una por núcleo
Sección crítica	taskENTER_CRITICAL()	taskENTER_CRITICAL(&mux)
Configuración	FreeRTOSConfig.h del proyecto	menuconfig (Kconfig)

Lo demás no cambia: **tareas, semáforos, mutex, colas y timers** usan la misma API que en la KL46Z.

DEMO 5 · EJEMPLOS DEL REPOSITORIO

Lo nuevo se ve en tres ejemplos

Ejemplo	Qué mirar en el monitor	Por qué en la KL46Z no pasa
2 · Dos tareas	El núcleo informado cambia entre mensajes	Hay un solo núcleo
4 · Mutex	USAR_MUTEX 0: espera ≈580 ms · 1: ≈80 ms	Pasa igual, pero en el S3 solo si las tareas comparten núcleo
6 · Cola + timer	El LED parpadea sin una tarea propia	La callback corre en Tmr Svc, en el núcleo 0

Los ejemplos 3 (semáforo) y 5 (cola) son repaso: el concepto es el mismo que en la KL46Z.

Ejemplos/<ejemplo>/blink_example_main.c → main/ · Full Clean · Build, Flash and Monitor

PARA LLEVARSE

El botón Build es gcc, un linker script y una docena de flags.

Un SDK como ESP-IDF hace lo mismo, a escala: CMake, componentes y Kconfig.

FreeRTOS es el mismo: en el ESP32-S3 cambian el arranque y los dos núcleos.

github.com/sea2-dsi-fceia/FreeRTOS_ESP32-S3

MAPA DE MEMORIA

Dónde quedó cada cosa del blinky

FLASH · 256 KB

0x00000000	.isr_vector · 192 B
0x00000400	.flash_config · 16 B
0x00000410	.text · código
0x000004BC	valores iniciales de .data
…	libre

Reset_Handler
copia al arrancar

RAM · 32 KB

0x1FFFE000	.data · demora_ciclos
0x1FFFE004	.bss · contador
0x1FFFE008	.stack · 1 KB reservado
…	libre
0x20006000	_estack · tope

SECCIONES

Cada variable va a una sección distinta

Sección	En el blinky	Dónde vive	Quién la inicializa
.text	main(), Reset_Handler	FLASH	El grabador
.rodata	constantes, strings	FLASH	El grabador
.data	demora_ciclos = 300000	RAM + copia en FLASH	Reset_Handler copia
.bss	contador_parpadeos	RAM	Reset_Handler pone en 0
.stack	1 KB reservado	RAM	Nadie: lo usa la CPU

Una variable global con valor inicial ocupa **FLASH y RAM**. Una en cero, solo RAM.

ANÁLISIS DEL BINARIO

Los flags se miden en bytes

Variante (arm-none-eabi-size)	text	data	bss
Release -Os + --gc-sections	380	4	1028
Release sin --gc-sections	392	4	1028
Debug -O0 + --gc-sections	472	4	1028

+12 B sin --gc-sections: es funcion_no_usada(), que nadie llama.

FLASH = text + data · **RAM** = data + bss. Los 1028 B de bss incluyen el stack de 1 KB.

ANÁLISIS DEL BINARIO

Cuatro herramientas para mirar adentro del .elf

size

¿Cuánta FLASH y RAM uso?

nm -S --size-sort

¿Qué símbolo ocupa más? ¿En qué dirección quedó?

objdump -d / -h

Desensamblado y tabla de secciones con LMA y VMA.

-Wl,-Map=blinky.map

De qué .o salió cada byte y qué descartó el linker.

MAKE

El Makefile del blinky

```
OBJETOS = $(addprefix $(BUILD)/,$(FUENTES:.c=.o))

$(BUILD)/blinky.elf: $(OBJETOS) $(LDSCRIPT)
    $(CC) $(LDFLAGS) $(OBJETOS) -o $@

$(BUILD)/%.o: %.c Makefile
    @cmake -E make_directory $(dir $@)
    $(CC) $(CFLAGS) -c $< -o $@

-include $(OBJETOS:.o=.d)
```

%.o: %.c
regla patrón: una para todos los .c

$@ · $<
objetivo · primer prerequisito

-MMD -MP
gcc genera los .d con los headers

$(LDSCRIPT)
si cambia el .ld, se relinkea

CMAKE + NINJA

El toolchain file: compilar para otro procesador

```
# cmake/arm-none-eabi.cmake
set(CMAKE_SYSTEM_NAME Generic)
set(CMAKE_SYSTEM_PROCESSOR arm)

set(CMAKE_C_COMPILER arm-none-eabi-gcc)
set(CMAKE_OBJCOPY arm-none-eabi-objcopy)
set(CMAKE_SIZE arm-none-eabi-size)

set(CMAKE_TRY_COMPILE_TARGET_TYPE
    STATIC_LIBRARY)
```

Generic

Sin sistema operativo: no busca librerías del host.

TRY_COMPILE

CMake prueba el compilador armando un ejecutable. Sin linker script fallaría; así prueba con una librería.

Separado

El mismo CMakeLists sirve para otro toolchain.

PANORAMA

Un fuente, muchas variantes de firmware

Tipo de build

Debug: -O0 -g3, para depurar paso a paso.

Release: -Os, para producción.

Un directorio de build por cada uno.

Opciones

Variables del build que llegan al código como #define.

-DUSAR_UART=1

Placa, periféricos, nivel de log.

Kconfig

El sistema de Linux y Zephyr: menuconfig genera un .config.

#if CONFIG_GPIO

Escala a cientos de opciones con dependencias.

PANORAMA

"En mi máquina compila" no alcanza

Otra versión de GCC genera otro binario. El build se versiona como el código: **toolchain fijo** (contenedor o devcontainer) y **CI** que compila en cada push.

```
# .github/workflows/build.yml
on: push
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: sudo apt-get install -y gcc-arm-none-eabi ninja-build
      - run: cmake -B build -G Ninja -DCMAKE_TOOLCHAIN_FILE=cmake/arm-none-eabi.cmake
      - run: cmake --build build
```

EJERCICIOS

Para hacer con el ejemplo

1. Separar el manejo del LED en **led.c / led.h** y sumarlo al Makefile y al CMakeLists.

2. Achicar la RAM a **1K** en el linker script y explicar el error del linker.

3. Compilar sin **-nostdlib**. ¿De dónde sale __bss_start__?

4. En el desensamblado de **Reset_Handler**, encontrar los lazos de .data y .bss.